SYNAPSE Implementation Guide

AI-Powered Technology Intelligence Platform

Development Team

March 7, 2026

Contents

1 Development Environment Setup **2**
1.1 Prerequisites . . . 2
1.2 Repository Structure . . . 2
1.3 Python Virtual Environment Setup . . . 3
1.4 Backend Dependencies . . . 3
1.5 Frontend Dependencies . . . 4
1.6 Environment Variables Configuration . . . 4
1.7 Docker Compose Setup . . . 5

2 Backend Implementation **8**
2.1 Django Project Structure . . . 8
2.2 Django Models . . . 10
2.3 REST Framework Serializers . . . 11
2.4 JWT Authentication Setup . . . 12
2.5 Celery Configuration . . . 13
2.6 FastAPI AI Service . . . 13

3 Database Setup **15**
3.1 PostgreSQL Installation and Configuration . . . 15
3.2 pgvector Extension Setup . . . 15
3.3 Django Migrations . . . 16
3.4 Redis Configuration . . . 16

4 Web Scraping Implementation **17**
4.1 Scrapy Project Structure . . . 17
4.2 HackerNews Spider . . . 17
4.3 GitHub Trending Spider . . . 18
4.4 arXiv Spider . . . 18
4.5 Scrapy Pipeline and Settings . . . 19
4.6 Celery Beat Scheduled Tasks . . . 20

5 NLP Pipeline Implementation **22**
5.1 Text Processing and spaCy Setup . . . 22
5.2 Summarization with Transformers . . . 23
5.3 Sentiment Analysis . . . 23
5.4 Embedding Generation . . . 24

6 Vector Database Implementation **25**
6.1 pgvector Configuration . . . 25
6.2 Vector Similarity Search . . . 25

7 AI Chat Assistant (RAG) Implementation **27**
7.1 RAG Pipeline Setup . . . 27
7.2 FastAPI Chat Endpoint . . . 28

8 Agentic AI Implementation **30**
8.1 LangChain Agent Setup . . . 30

9 Frontend Implementation **32**
9.1 Next.js Project Setup . . . 32
9.2 API Client Configuration . . . 32
9.3 State Management with Zustand . . . 33
9.4 Pages Structure . . . 34

10 Automation System Implementation **35**
10.1 Workflow Model and Task Queue . . . 35
10.2 Celery Workflow Execution . . . 36

11 Testing Strategy **38**
11.1 Backend Unit Tests . . . 38
11.2 API Integration Tests . . . 38
11.3 Frontend Tests with Jest . . . 39

12 Code Quality and Deployment **41**
12.1 Linting and Formatting . . . 41
12.2 Pre-commit Hooks . . . 41
12.3 Continuous Integration with GitHub Actions . . . 42
12.4 Deployment Checklist . . . 43

13 Monitoring and Observability **44**
13.1 Prometheus Metrics . . . 44

Chapter 1

Development Environment Setup

1.1 Prerequisites

Before starting SYNAPSE development, ensure you have the following installed:

- **Python 3.11** or higher
- **Node.js 20** LTS with npm/yarn
- **Docker** and **Docker Compose** (v2.0+)
- **Git** (v2.30+)
- **PostgreSQL 15** client tools
- **Redis 7**
- Text editor or IDE (VSCode, PyCharm, WebStorm recommended)

Verify installations:

```
python3 --version # Should be 3.11+
node --version # Should be 20+
docker --version
git --version
pg_config --version
redis-cli --version
```

Listing 1.1: Verify prerequisites

1.2 Repository Structure

SYNAPSE uses a monorepo structure for unified version control and dependency management:

```
synapse/
+-- backend/ # Django + FastAPI backend
| +-- manage.py
| +-- synapse/ # Main Django project
| | +-- settings.py
| | +-- urls.py
| | +-- wsgi.py
| | \`-- asgi.py
| +-- apps/
| | +-- core/
```

```
| | +-- users/
| | +-- articles/
| | +-- repositories/
| | +-- papers/
| | +-- videos/
| | +-- automation/
| | +-- agents/
| | \`-- documents/
| +-- ai_service/ # FastAPI AI service
| | +-- main.py
| | \`-- routes/
| +-- requirements.txt
| \`-- Dockerfile
+-- frontend/ # Next.js React application
| +-- pages/
| +-- components/
| +-- styles/
| +-- package.json
| \`-- Dockerfile
+-- scraper/ # Scrapy spider project
| +-- scrapy.cfg
| +-- synapse_scraper/
| | +-- spiders/
| | +-- items.py
| | +-- pipelines.py
| | \`-- settings.py
| \`-- requirements.txt
+-- ai_engine/ # NLP and ML processing
| +-- nlp_pipeline.py
| +-- embeddings.py
| +-- requirements.txt
| \`-- Dockerfile
+-- docs/ # Documentation
\`-- docker-compose.yml
```

Listing 1.2: Repository structure

1.3 Python Virtual Environment Setup

Navigate to the backend directory and create a virtual environment:

```
cd synapse/backend
python3.11 -m venv venv
source venv/bin/activate # On Windows: venv\Scripts\activate

# Upgrade pip and setuptools
pip install --upgrade pip setuptools wheel
```

Listing 1.3: Setup Python virtual environment

1.4 Backend Dependencies

```
Django==4.2.7
djangorestframework==3.14.0
djangorestframework-simplejwt==5.3.2
django-cors-headers==4.3.1
```

```
django-environ==0.21.0
django-celery-beat==2.5.0
django-celery-results==2.5.1
psycopg2-binary==2.9.9
redis==5.0.1
celery==5.3.4
fastapi==0.104.1
uvicorn==0.24.0
sqlalchemy==2.0.23
langchain==0.1.3
openai==1.3.5
scrapy==2.11.0
beautifulsoup4==4.12.2
playwright==1.40.0
spacy==3.7.2
transformers==4.35.2
sentence-transformers==2.2.2
pinecone-client==3.0.0
pytest==7.4.3
pytest-django==4.7.0
```

Listing 1.4: Backend requirements.txt excerpt

Install all dependencies:

```
pip install -r requirements.txt
python -m spacy download en_core_web_sm
```

Listing 1.5: Install backend dependencies

1.5 Frontend Dependencies

```
cd ../frontend
npm install
# or
yarn install
```

Listing 1.6: Install frontend dependencies

1.6 Environment Variables Configuration

Create a `.env` file in the backend directory with all required configurations:

```
# Django Settings
DEBUG=False
SECRET_KEY=your-super-secret-key-change-in-production
ALLOWED_HOSTS=localhost,127.0.0.1,synapse.local

# Database
DATABASE_URL=postgresql://synapse_user:password@localhost:5432/synapse_db
POSTGRES_USER=synapse_user
POSTGRES_PASSWORD=secure_password
POSTGRES_DB=synapse_db

# Redis
REDIS_URL=redis://localhost:6379/0
CELERY_BROKER_URL=redis://localhost:6379/1
```

```
CELERY_RESULT_BACKEND=redis://localhost:6379/2

# OpenAI
OPENAI_API_KEY=sk-your-api-key-here

# Other API Keys
GITHUB_TOKEN=ghp_your_github_token
YOUTUBE_API_KEY=your_youtube_api_key
PINECONE_API_KEY=your_pinecone_key
PINECONE_ENVIRONMENT=your_environment

# AWS
AWS_ACCESS_KEY_ID=your_access_key
AWS_SECRET_ACCESS_KEY=your_secret_key
AWS_STORAGE_BUCKET_NAME=synapse-bucket
AWS_S3_REGION_NAME=us-east-1

# Email
SENDGRID_API_KEY=your_sendgrid_key
EMAIL_FROM=noreply@synapse.ai

# CORS
CORS_ALLOWED_ORIGINS=http://localhost:3000,http://localhost:8000

# JWT
JWT_SECRET_KEY=your-jwt-secret-key
JWT_ALGORITHM=HS256
JWT_EXPIRATION_HOURS=24
```

Listing 1.7: Backend .env file

Create `.env.local` for frontend:

```
NEXT_PUBLIC_API_URL=http://localhost:8000/api
NEXT_PUBLIC_WS_URL=ws://localhost:8000/ws
NEXT_PUBLIC_APP_NAME=SYNAPSE
NEXT_PUBLIC_APP_VERSION=1.0.0
```

Listing 1.8: Frontend .env.local file

1.7 Docker Compose Setup

Create `docker-compose.yml` at the project root:

```yaml
version: '3.8'

services:
  postgres:
    image: postgres:15-alpine
    container_name: synapse_postgres
    environment:
      POSTGRES_USER: ${POSTGRES_USER}
      POSTGRES_PASSWORD: ${POSTGRES_PASSWORD}
      POSTGRES_DB: ${POSTGRES_DB}
    ports:
      - "5432:5432"
    volumes:
      - postgres_data:/var/lib/postgresql/data
    healthcheck:
```

```yaml
    test: ["CMD-SHELL", "pg_isready -U ${POSTGRES_USER}"]
    interval: 10s
    timeout: 5s
    retries: 5

  redis:
    image: redis:7-alpine
    container_name: synapse_redis
    ports:
      - "6379:6379"
    volumes:
      - redis_data:/data
    healthcheck:
      test: ["CMD", "redis-cli", "ping"]
      interval: 10s
      timeout: 5s
      retries: 5

  backend:
    build:
      context: ./backend
      dockerfile: Dockerfile
    container_name: synapse_backend
    command: >
      sh -c "python manage.py migrate &&
             python manage.py collectstatic --noinput &&
             gunicorn synapse.wsgi:application --bind 0.0.0.0:8000"
    ports:
      - "8000:8000"
    environment:
      - DEBUG=${DEBUG}
      - SECRET_KEY=${SECRET_KEY}
      - DATABASE_URL=postgresql://${POSTGRES_USER}:${POSTGRES_PASSWORD}@postgres:5432/${POSTGRES_DB}
      - REDIS_URL=redis://redis:6379/0
      - CELERY_BROKER_URL=redis://redis:6379/1
    depends_on:
      postgres:
        condition: service_healthy
      redis:
        condition: service_healthy
    volumes:
      - ./backend:/app

  celery:
    build:
      context: ./backend
      dockerfile: Dockerfile
    container_name: synapse_celery
    command: celery -A synapse worker -l info
    environment:
      - DEBUG=${DEBUG}
      - DATABASE_URL=postgresql://${POSTGRES_USER}:${POSTGRES_PASSWORD}@postgres:5432/${POSTGRES_DB}
      - REDIS_URL=redis://redis:6379/0
      - CELERY_BROKER_URL=redis://redis:6379/1
    depends_on:
      - postgres
      - redis
    volumes:
      - ./backend:/app
```

```yaml
  celery_beat:
    build:
      context: ./backend
      dockerfile: Dockerfile
    container_name: synapse_celery_beat
    command: celery -A synapse beat -l info --scheduler django_celery_beat.schedulers:
        DatabaseScheduler
    environment:
      - DEBUG=${DEBUG}
      - DATABASE_URL=postgresql://${POSTGRES_USER}:${POSTGRES_PASSWORD}@postgres:5432/${POSTGRES_DB}
      - REDIS_URL=redis://redis:6379/0
      - CELERY_BROKER_URL=redis://redis:6379/1
    depends_on:
      - postgres
      - redis
    volumes:
      - ./backend:/app

  frontend:
    build:
      context: ./frontend
      dockerfile: Dockerfile
    container_name: synapse_frontend
    ports:
      - "3000:3000"
    environment:
      - NEXT_PUBLIC_API_URL=http://backend:8000/api
    depends_on:
      - backend
    volumes:
      - ./frontend:/app

volumes:
  postgres_data:
  redis_data:
```

Listing 1.9: Docker Compose configuration

Start all services:

```bash
docker-compose up -d
docker-compose logs -f backend # Monitor backend logs
```

Listing 1.10: Start Docker services

Chapter 2

Backend Implementation

2.1 Django Project Structure

Initialize Django project:

```
cd backend
django-admin startproject synapse .
python manage.py startapp core
python manage.py startapp users
python manage.py startapp articles
python manage.py startapp repositories
python manage.py startapp papers
python manage.py startapp videos
python manage.py startapp automation
python manage.py startapp agents
python manage.py startapp documents
```

Listing 2.1: Create Django project

Update `synapse/settings.py`:

```
import os
from pathlib import Path
from datetime import timedelta
import environ

env = environ.Env()
env.read_env()

BASE_DIR = Path(__file__).resolve().parent.parent

SECRET_KEY = env('SECRET_KEY')
DEBUG = env.bool('DEBUG', False)
ALLOWED_HOSTS = env.list('ALLOWED_HOSTS')

INSTALLED_APPS = [
    'django.contrib.admin',
    'django.contrib.auth',
    'django.contrib.contenttypes',
    'django.contrib.sessions',
    'django.contrib.messages',
    'django.contrib.staticfiles',
    'rest_framework',
    'rest_framework_simplejwt',
    'corsheaders',
```

```
    'django_celery_beat',
    'django_celery_results',
    'core',
    'users',
    'articles',
    'repositories',
    'papers',
    'videos',
    'automation',
    'agents',
    'documents',
]

MIDDLEWARE = [
    'django.middleware.security.SecurityMiddleware',
    'corsheaders.middleware.CorsMiddleware',
    'django.middleware.common.CommonMiddleware',
    'django.middleware.csrf.CsrfViewMiddleware',
    'django.contrib.auth.middleware.AuthenticationMiddleware',
    'django.contrib.messages.middleware.MessageMiddleware',
]

REST_FRAMEWORK = {
    'DEFAULT_AUTHENTICATION_CLASSES': (
        'rest_framework_simplejwt.authentication.JWTAuthentication',
    ),
    'DEFAULT_PERMISSION_CLASSES': [
        'rest_framework.permissions.IsAuthenticated',
    ],
    'DEFAULT_PAGINATION_CLASS': 'rest_framework.pagination.PageNumberPagination',
    'PAGE_SIZE': 20,
    'DEFAULT_FILTER_BACKENDS': [
        'rest_framework.filters.SearchFilter',
        'rest_framework.filters.OrderingFilter',
    ],
}

SIMPLE_JWT = {
    'ACCESS_TOKEN_LIFETIME': timedelta(hours=24),
    'REFRESH_TOKEN_LIFETIME': timedelta(days=7),
    'ALGORITHM': 'HS256',
    'SIGNING_KEY': env('SECRET_KEY'),
}

DATABASES = {
    'default': {
        'ENGINE': 'django.db.backends.postgresql',
        'NAME': env('POSTGRES_DB'),
        'USER': env('POSTGRES_USER'),
        'PASSWORD': env('POSTGRES_PASSWORD'),
        'HOST': env('DATABASE_HOST', 'localhost'),
        'PORT': env('DATABASE_PORT', '5432'),
        'CONN_MAX_AGE': 600,
        'OPTIONS': {
            'connect_timeout': 10,
        }
    }
}
```

```
CELERY_BROKER_URL = env('CELERY_BROKER_URL')
CELERY_RESULT_BACKEND = env('CELERY_RESULT_BACKEND')
CELERY_ACCEPT_CONTENT = ['json']
CELERY_TASK_SERIALIZER = 'json'
CELERY_RESULT_SERIALIZER = 'json'

CORS_ALLOWED_ORIGINS = env.list('CORS_ALLOWED_ORIGINS')

STATIC_URL = '/static/'
STATIC_ROOT = BASE_DIR / 'staticfiles'
MEDIA_URL = '/media/'
MEDIA_ROOT = BASE_DIR / 'media'

AUTH_USER_MODEL = 'users.User'
```

Listing 2.2: Django settings configuration

2.2 Django Models

Create `users/models.py`:

```
from django.contrib.auth.models import AbstractUser
from django.db import models
import uuid

class User(AbstractUser):
    id = models.UUIDField(primary_key=True, default=uuid.uuid4, editable=False)
    email = models.EmailField(unique=True)
    organization = models.CharField(max_length=255, blank=True)
    avatar_url = models.URLField(blank=True)
    bio = models.TextField(blank=True)
    preferences = models.JSONField(default=dict)
    is_premium = models.BooleanField(default=False)
    created_at = models.DateTimeField(auto_now_add=True)
    updated_at = models.DateTimeField(auto_now=True)

    class Meta:
        ordering = ['-created_at']
        indexes = [
            models.Index(fields=['email']),
            models.Index(fields=['is_premium']),
        ]

    def __str__(self):
        return self.email
```

Listing 2.3: User model definition

Create `articles/models.py`:

```
from django.db import models
from django.contrib.postgres.search import SearchVectorField
from pgvector.django import VectorField
import uuid

class Article(models.Model):
    SOURCE_CHOICES = [
        ('hackernews', 'Hacker News'),
        ('medium', 'Medium'),
```

```
        ('dev_to', 'Dev.to'),
        ('other', 'Other'),
    ]

    id = models.UUIDField(primary_key=True, default=uuid.uuid4)
    title = models.CharField(max_length=500, db_index=True)
    content = models.TextField()
    source = models.CharField(max_length=50, choices=SOURCE_CHOICES)
    source_url = models.URLField(unique=True)
    author = models.CharField(max_length=255, blank=True)
    published_date = models.DateTimeField()
    created_at = models.DateTimeField(auto_now_add=True)
    updated_at = models.DateTimeField(auto_now=True)

    # NLP fields
    summary = models.TextField(blank=True)
    keywords = models.JSONField(default=list)
    sentiment_score = models.FloatField(null=True)
    topic_category = models.CharField(max_length=100, blank=True, db_index=True)

    # Vector embeddings
    embedding = VectorField(dimensions=1536, null=True)

    # Metadata
    word_count = models.IntegerField(default=0)
    read_time_minutes = models.IntegerField(default=0)
    is_archived = models.BooleanField(default=False)

    class Meta:
        ordering = ['-published_date']
        indexes = [
            models.Index(fields=['topic_category']),
            models.Index(fields=['published_date']),
            models.Index(fields=['source']),
        ]

    def __str__(self):
        return self.title
```

Listing 2.4: Article model definition

2.3 REST Framework Serializers

Create `users/serializers.py`:

```
from rest_framework import serializers
from .models import User

class UserSerializer(serializers.ModelSerializer):
    class Meta:
        model = User
        fields = ['id', 'email', 'first_name', 'last_name', 'organization',
                 'avatar_url', 'bio', 'preferences', 'is_premium', 'created_at']
        read_only_fields = ['id', 'created_at']

class UserUpdateSerializer(serializers.ModelSerializer):
    class Meta:
        model = User
```

```
        fields = ['first_name', 'last_name', 'organization', 'bio',
                 'avatar_url', 'preferences']
```

Listing 2.5: User serializer

Create `articles/serializers.py`:

```
from rest_framework import serializers
from .models import Article

class ArticleListSerializer(serializers.ModelSerializer):
    class Meta:
        model = Article
        fields = ['id', 'title', 'source', 'published_date',
                 'author', 'summary', 'topic_category', 'read_time_minutes']

class ArticleDetailSerializer(serializers.ModelSerializer):
    class Meta:
        model = Article
        fields = '__all__'
        read_only_fields = ['id', 'created_at', 'updated_at', 'embedding']
```

Listing 2.6: Article serializer

2.4 JWT Authentication Setup

Create `users/views.py`:

```
from rest_framework import status, views
from rest_framework.response import Response
from rest_framework_simplejwt.views import TokenObtainPairView
from rest_framework_simplejwt.serializers import TokenObtainPairSerializer
from .serializers import UserSerializer, UserUpdateSerializer
from .models import User

class CustomTokenObtainPairSerializer(TokenObtainPairSerializer):
    @classmethod
    def get_token(cls, user):
        token = super().get_token(user)
        token['email'] = user.email
        token['organization'] = user.organization
        return token

class CustomTokenObtainPairView(TokenObtainPairView):
    serializer_class = CustomTokenObtainPairSerializer

class RegisterView(views.APIView):
    permission_classes = []

    def post(self, request):
        serializer = UserSerializer(data=request.data)
        if serializer.is_valid():
            user = User.objects.create_user(
                email=request.data['email'],
                password=request.data['password'],
                first_name=request.data.get('first_name', ''),
                last_name=request.data.get('last_name', '')
            )
            return Response(serializer.data, status=status.HTTP_201_CREATED)
```

```
        return Response(serializer.errors, status=status.HTTP_400_BAD_REQUEST)

class ProfileView(views.APIView):
    def get(self, request):
        serializer = UserSerializer(request.user)
        return Response(serializer.data)

    def put(self, request):
        serializer = UserUpdateSerializer(request.user, data=request.data, partial=True)
        if serializer.is_valid():
            serializer.save()
            return Response(serializer.data)
        return Response(serializer.errors, status=status.HTTP_400_BAD_REQUEST)
```

Listing 2.7: JWT authentication views

2.5 Celery Configuration

Create `synapse/celery.py`:

```
import os
from celery import Celery

os.environ.setdefault('DJANGO_SETTINGS_MODULE', 'synapse.settings')

app = Celery('synapse')
app.config_from_object('django.conf:settings', namespace='CELERY')
app.autodiscover_tasks()

@app.task(bind=True)
def debug_task(self):
    print(f'Request: {self.request!r}')
```

Listing 2.8: Celery configuration

Create `synapse/__init__.py`:

```
from .celery import app as celery_app

__all__ = ('celery_app',)
```

Listing 2.9: Celery initialization in Django

2.6 FastAPI AI Service

Create `ai_service/main.py`:

```
from fastapi import FastAPI, HTTPException
from fastapi.middleware.cors import CORSMiddleware
from pydantic import BaseModel
from typing import List, Optional
import os

app = FastAPI(title="SYNAPSE AI Service", version="1.0.0")

# CORS configuration
origins = os.getenv('CORS_ALLOWED_ORIGINS', 'http://localhost:3000').split(',')
app.add_middleware(
```

```python
    CORSMiddleware,
    allow_origins=origins,
    allow_credentials=True,
    allow_methods=["*"],
    allow_headers=["*"],
)

class ChatRequest(BaseModel):
    message: str
    conversation_id: Optional[str] = None
    context: Optional[List[str]] = None

class ChatResponse(BaseModel):
    response: str
    conversation_id: str
    sources: List[str] = []

@app.get("/health")
async def health_check():
    return {"status": "healthy"}

@app.post("/chat", response_model=ChatResponse)
async def chat(request: ChatRequest):
    """AI Chat endpoint with RAG capabilities"""
    try:
        # RAG pipeline implementation
        # This will be filled in the RAG section
        return ChatResponse(
            response="Response generated by RAG pipeline",
            conversation_id=request.conversation_id or "new_conv"
        )
    except Exception as e:
        raise HTTPException(status_code=500, detail=str(e))

if __name__ == "__main__":
    import uvicorn
    uvicorn.run(app, host="0.0.0.0", port=8001)
```

Listing 2.10: FastAPI AI service setup

Chapter 3

Database Setup

3.1 PostgreSQL Installation and Configuration

Install PostgreSQL 15:

```
# macOS with Homebrew
brew install postgresql@15
brew services start postgresql@15

# Ubuntu/Debian
sudo apt-get install postgresql-15
sudo systemctl start postgresql

# Verify installation
psql --version
```

Listing 3.1: PostgreSQL installation

Create database and user:

```
sudo -u postgres psql

CREATE USER synapse_user WITH PASSWORD 'secure_password';
CREATE DATABASE synapse_db OWNER synapse_user;
ALTER ROLE synapse_user SET client_encoding TO 'utf8';
ALTER ROLE synapse_user SET default_transaction_isolation TO 'read committed';
ALTER ROLE synapse_user SET default_transaction_deferrable TO on;
ALTER ROLE synapse_user SET timezone TO 'UTC';
GRANT ALL PRIVILEGES ON DATABASE synapse_db TO synapse_user;
```

Listing 3.2: PostgreSQL setup

3.2 pgvector Extension Setup

Install pgvector for vector similarity search:

```
# Clone and build pgvector
git clone https://github.com/pgvector/pgvector.git
cd pgvector
make
sudo make install

# In PostgreSQL
```

```
psql -U synapse_user -d synapse_db

CREATE EXTENSION IF NOT EXISTS vector;
```

Listing 3.3: pgvector installation

3.3 Django Migrations

Run Django migrations to create tables:

```
cd backend
python manage.py makemigrations
python manage.py migrate

# Create superuser
python manage.py createsuperuser
```

Listing 3.4: Django migrations

3.4 Redis Configuration

Start Redis server:

```
# macOS
brew install redis
brew services start redis

# Ubuntu/Debian
sudo apt-get install redis-server
sudo systemctl start redis-server

# Verify connection
redis-cli ping # Should return PONG
```

Listing 3.5: Redis setup

Chapter 4

Web Scraping Implementation

4.1 Scrapy Project Structure

Initialize Scrapy project in the scraper directory:

```
cd scraper
scrapy startproject synapse_scraper .
scrapy genspider hackernews hacker-news.firebaseio.com
scrapy genspider github api.github.com
scrapy genspider arxiv export.arxiv.org
scrapy genspider youtube youtube.com
```

Listing 4.1: Scrapy project setup

4.2 HackerNews Spider

Create `scraper/synapse_scraper/spiders/hackernews.py`:

```
import scrapy
import json
from datetime import datetime
import requests

class HackernewsSpider(scrapy.Spider):
    name = 'hackernews'
    allowed_domains = ['hacker-news.firebaseio.com']
    start_urls = ['https://hacker-news.firebaseio.com/v0/topstories.json']

    def parse(self, response):
        story_ids = json.loads(response.text)[:30] # Top 30 stories
        for story_id in story_ids:
            url = f'https://hacker-news.firebaseio.com/v0/item/{story_id}.json'
            yield scrapy.Request(url, callback=self.parse_story)

    def parse_story(self, response):
        data = json.loads(response.text)
        if data.get('type') == 'story':
            yield {
                'title': data.get('title'),
                'url': data.get('url'),
                'author': data.get('by'),
                'score': data.get('score'),
                'timestamp': datetime.fromtimestamp(data.get('time')),
```

```
            'source': 'hackernews',
        }
```

Listing 4.2: HackerNews spider

4.3 GitHub Trending Spider

Create `scraper/synapse_scraper/spiders/github.py`:

```
import scrapy
import requests
from datetime import datetime, timedelta

class GithubSpider(scrapy.Spider):
    name = 'github'

    def __init__(self, *args, **kwargs):
        super(GithubSpider, self).__init__(*args, **kwargs)
        self.github_token = self.settings.get('GITHUB_TOKEN')

    def start_requests(self):
        # Fetch trending repositories from past week
        since_date = (datetime.now() - timedelta(days=7)).isoformat()
        url = f'https://api.github.com/search/repositories?q=created:>{since_date}&sort=stars&order=
            desc&per_page=30'

        headers = {'Authorization': f'token {self.github_token}'}
        yield scrapy.Request(url, headers=headers, callback=self.parse)

    def parse(self, response):
        data = response.json()
        for repo in data.get('items', []):
            yield {
                'name': repo['name'],
                'url': repo['html_url'],
                'description': repo['description'],
                'stars': repo['stargazers_count'],
                'language': repo['language'],
                'topics': repo.get('topics', []),
                'source': 'github',
            }
```

Listing 4.3: GitHub trending spider

4.4 arXiv Spider

Create `scraper/synapse_scraper/spiders/arxiv.py`:

```
import scrapy
import feedparser
from datetime import datetime, timedelta

class ArxivSpider(scrapy.Spider):
    name = 'arxiv'

    def start_requests(self):
        # Recent papers in Computer Science
```

```
        base_url = 'http://export.arxiv.org/api/query?'
        query = 'cat:cs.AI AND submittedDate:[202301010000 TO 202401010000]'
        params = {'search_query': query, 'start': 0, 'max_results': 30}

        url = base_url + '&'.join([f'{k}={v}' for k, v in params.items()])
        yield scrapy.Request(url, callback=self.parse)

    def parse(self, response):
        feed = feedparser.parse(response.text)
        for entry in feed.entries:
            yield {
                'title': entry.title,
                'authors': [author.name for author in entry.authors],
                'summary': entry.summary,
                'url': entry.id,
                'published': entry.published,
                'arxiv_id': entry.id.split('/abs/')[-1],
                'source': 'arxiv',
            }
```

Listing 4.4: arXiv spider

4.5 Scrapy Pipeline and Settings

Create `scraper/synapse_scraper/pipelines.py`:

```
import hashlib
from itemadapter import ItemAdapter
from scrapy.exceptions import DropItem
import psycopg2

class DuplicatesPipeline:
    def __init__(self):
        self.ids_seen = set()

    def process_item(self, item, spider):
        adapter = ItemAdapter(item)
        url = adapter.get('url')

        # Create hash of URL for deduplication
        url_hash = hashlib.md5(url.encode()).hexdigest()

        if url_hash in self.ids_seen:
            raise DropItem(f"Duplicate item found: {url}")
        else:
            self.ids_seen.add(url_hash)
            return item

class PostgresPipeline:
    def __init__(self, db_config):
        self.db_config = db_config
        self.conn = None

    @classmethod
    def from_crawler(cls, crawler):
        return cls(db_config=crawler.settings.get('DB_CONFIG'))

    def open_spider(self, spider):
```

```
        self.conn = psycopg2.connect(**self.db_config)

    def close_spider(self, spider):
        self.conn.close()

    def process_item(self, item, spider):
        cursor = self.conn.cursor()
        # Insert or update item in database
        cursor.execute(
            """INSERT INTO articles_article
               (title, content, source, source_url, author, published_date, created_at)
               VALUES (%s, %s, %s, %s, %s, %s, NOW())
               ON CONFLICT (source_url) DO NOTHING""",
            (item['title'], item.get('summary', ''), item['source'],
             item['url'], item.get('author', ''), item.get('timestamp'))
        )
        self.conn.commit()
        return item
```

Listing 4.5: Scrapy pipeline for data deduplication

Update `scraper/synapse_scraper/settings.py`:

```
BOT_NAME = 'synapse_scraper'
SPIDER_MODULES = ['synapse_scraper.spiders']
NEWSPIDER_MODULE = 'synapse_scraper.spiders'

ROBOTSTXT_OBEY = True
CONCURRENT_REQUESTS = 16
DOWNLOAD_DELAY = 2

ITEM_PIPELINES = {
    'synapse_scraper.pipelines.DuplicatesPipeline': 300,
    'synapse_scraper.pipelines.PostgresPipeline': 400,
}

DB_CONFIG = {
    'host': 'localhost',
    'database': 'synapse_db',
    'user': 'synapse_user',
    'password': 'secure_password',
}

GITHUB_TOKEN = 'your_github_token_here'
```

Listing 4.6: Scrapy settings configuration

4.6 Celery Beat Scheduled Tasks

Create `backend/synapse/celery_tasks.py`:

```
from celery import shared_task
from celery.schedules import crontab
from django.core.management import call_command
import subprocess

@shared_task
def run_hackernews_spider():
    """Run HackerNews spider every 6 hours"""
```

```python
    subprocess.run(['scrapy', 'crawl', 'hackernews'], cwd='../scraper')

@shared_task
def run_github_spider():
    """Run GitHub spider daily"""
    subprocess.run(['scrapy', 'crawl', 'github'], cwd='../scraper')

@shared_task
def run_arxiv_spider():
    """Run arXiv spider daily"""
    subprocess.run(['scrapy', 'crawl', 'arxiv'], cwd='../scraper')

# Schedule in synapse/settings.py
CELERY_BEAT_SCHEDULE = {
    'run-hackernews-spider': {
        'task': 'synapse.celery_tasks.run_hackernews_spider',
        'schedule': crontab(minute=0, hour='*/6'),
    },
    'run-github-spider': {
        'task': 'synapse.celery_tasks.run_github_spider',
        'schedule': crontab(minute=0, hour=0),
    },
    'run-arxiv-spider': {
        'task': 'synapse.celery_tasks.run_arxiv_spider',
        'schedule': crontab(minute=0, hour=1),
    },
}
```

Listing 4.7: Celery scheduled scraping tasks

Chapter 5

NLP Pipeline Implementation

5.1 Text Processing and spaCy Setup

Create ai_engine/nlp_pipeline.py:

```
import spacy
import re
from typing import List, Dict

class NLPPipeline:
    def __init__(self):
        self.nlp = spacy.load('en_core_web_sm')

    def clean_text(self, text: str) -> str:
        """Clean and normalize text"""
        text = re.sub(r'http\S+|www\S+', '', text)
        text = re.sub(r'[^\w\s.]', '', text)
        text = re.sub(r'\s+', ' ', text).strip()
        return text

    def extract_keywords(self, text: str, top_n: int = 10) -> List[str]:
        """Extract keywords using spaCy"""
        doc = self.nlp(text)
        keywords = [token.text.lower() for token in doc
                    if token.is_alpha and not token.is_stop]
        return list(set(keywords))[:top_n]

    def get_entities(self, text: str) -> Dict:
        """Extract named entities"""
        doc = self.nlp(text)
        entities = {}
        for ent in doc.ents:
            if ent.label_ not in entities:
                entities[ent.label_] = []
            entities[ent.label_].append(ent.text)
        return entities

    def categorize_text(self, text: str) -> str:
        """Simple topic categorization"""
        topics = {
            'AI/ML': ['machine learning', 'neural', 'deep learning'],
            'Web': ['web', 'javascript', 'react'],
            'DevOps': ['docker', 'kubernetes', 'devops'],
            'Database': ['database', 'sql', 'postgresql'],
```

```
        }
        text_lower = text.lower()
        for topic, keywords in topics.items():
            if any(kw in text_lower for kw in keywords):
                return topic
        return 'General'
```

Listing 5.1: NLP pipeline with spaCy

5.2 Summarization with Transformers

Create `ai_engine/summarizer.py`:

```
from transformers import pipeline
from typing import Optional

class TextSummarizer:
    def __init__(self):
        self.summarizer = pipeline(
            "summarization",
            model="facebook/bart-large-cnn",
            device=0
        )

    def summarize(self, text: str, max_length: int = 100) -> str:
        """Generate summary of text"""
        if len(text.split()) < 50:
            return text

        try:
            summary = self.summarizer(
                text,
                max_length=max_length,
                min_length=50,
                do_sample=False
            )
            return summary[0]['summary_text']
        except Exception as e:
            print(f"Summarization error: {e}")
            return text[:max_length]
```

Listing 5.2: Text summarization

5.3 Sentiment Analysis

Create `ai_engine/sentiment.py`:

```
from transformers import pipeline
import numpy as np

class SentimentAnalyzer:
    def __init__(self):
        self.classifier = pipeline(
            "sentiment-analysis",
            model="distilbert-base-uncased-finetuned-sst-2-english"
        )
```

```python
    def analyze(self, text: str) -> Dict[str, float]:
        """Analyze sentiment and return score"""
        result = self.classifier(text[:512])[0] # Limit to 512 tokens
        label = result['label']
        score = result['score']

        return {
            'sentiment': label,
            'score': score if label == 'POSITIVE' else -score
        }
```

Listing 5.3: Sentiment analysis

5.4 Embedding Generation

Create `ai_engine/embeddings.py`:

```python
from sentence_transformers import SentenceTransformer
import numpy as np
from typing import List, Union

class EmbeddingGenerator:
    def __init__(self, model_name: str = "all-MiniLM-L6-v2"):
        self.model = SentenceTransformer(model_name)

    def generate_embeddings(
        self,
        texts: Union[str, List[str]],
        normalize: bool = True
    ) -> np.ndarray:
        """Generate embeddings for text(s)"""
        if isinstance(texts, str):
            texts = [texts]

        embeddings = self.model.encode(texts, normalize_embeddings=normalize)
        return embeddings

    def get_embedding_dimension(self) -> int:
        """Return embedding dimension"""
        return self.model.get_sentence_embedding_dimension()
```

Listing 5.4: Embedding generation

Chapter 6

Vector Database Implementation

6.1 pgvector Configuration

Create a migration for vector field:

```
from django.db import migrations
from pgvector.django import VectorField

class Migration(migrations.Migration):
    dependencies = [
        ('articles', '0001_initial'),
    ]

    operations = [
        migrations.AddField(
            model_name='article',
            name='embedding',
            field=VectorField(dimensions=384, null=True),
        ),
    ]
```

Listing 6.1: pgvector migration

6.2 Vector Similarity Search

Create `backend/core/vector_search.py`:

```
from django.db.models import F
from pgvector.django import L2Distance, CosineDistance
from articles.models import Article

class VectorSearch:
    @staticmethod
    def semantic_search(query_embedding, top_k: int = 5):
        """Find similar articles using cosine distance"""
        results = Article.objects.annotate(
            distance=CosineDistance(F('embedding'), query_embedding)
        ).order_by('distance')[:top_k]
        return results

    @staticmethod
    def hybrid_search(query: str, query_embedding, top_k: int = 10):
        """Combine keyword and semantic search"""
```

```python
    from django.db.models import Q

    # Keyword search
    keyword_results = Article.objects.filter(
        Q(title__icontains=query) |
        Q(content__icontains=query)
    )

    # Vector search
    vector_results = Article.objects.annotate(
        distance=CosineDistance(F('embedding'), query_embedding)
    ).order_by('distance')[:top_k]

    # Combine and deduplicate
    combined = list(set(
        list(keyword_results[:5]) + list(vector_results[:5])
    ))
    return combined[:top_k]
```

Listing 6.2: Vector similarity search

Chapter 7

AI Chat Assistant (RAG) Implementation

7.1 RAG Pipeline Setup

Create ai_service/rag_pipeline.py:

```
from langchain.embeddings.openai import OpenAIEmbeddings
from langchain.vectorstores import PGVector
from langchain.chat_models import ChatOpenAI
from langchain.chains import ConversationalRetrievalChain
from langchain.memory import ConversationBufferMemory
import os

class RAGPipeline:
    def __init__(self):
        self.embeddings = OpenAIEmbeddings(
            openai_api_key=os.getenv('OPENAI_API_KEY')
        )

        self.vector_store = PGVector(
            connection_string=os.getenv('DATABASE_URL'),
            embedding_function=self.embeddings,
            table_name="articles_article"
        )

        self.llm = ChatOpenAI(
            openai_api_key=os.getenv('OPENAI_API_KEY'),
            model_name="gpt-3.5-turbo",
            temperature=0.7
        )

        self.memory = ConversationBufferMemory(
            memory_key="chat_history",
            return_messages=True
        )

    def create_chain(self):
        """Create RAG chain"""
        return ConversationalRetrievalChain.from_llm(
            llm=self.llm,
            retriever=self.vector_store.as_retriever(),
            memory=self.memory,
```

```
            return_source_documents=True,
            verbose=True
        )

    def query(self, question: str) -> Dict:
        """Query the RAG pipeline"""
        chain = self.create_chain()
        result = chain({"question": question})
        return {
            "answer": result['answer'],
            "sources": [doc.metadata for doc in result['source_documents']]
        }
```

Listing 7.1: LangChain RAG pipeline

7.2 FastAPI Chat Endpoint

Update `ai_service/main.py` with RAG:

```
from fastapi import FastAPI, WebSocket
from fastapi.responses import StreamingResponse
from rag_pipeline import RAGPipeline
import asyncio
import json

rag = RAGPipeline()

@app.post("/chat")
async def chat_endpoint(request: ChatRequest):
    """Chat with RAG-powered AI"""
    try:
        result = rag.query(request.message)
        return ChatResponse(
            response=result['answer'],
            conversation_id=request.conversation_id or "conv_id",
            sources=[str(s) for s in result['sources']]
        )
    except Exception as e:
        raise HTTPException(status_code=500, detail=str(e))

@app.websocket("/ws/chat")
async def websocket_chat(websocket: WebSocket):
    """WebSocket endpoint for streaming responses"""
    await websocket.accept()
    try:
        while True:
            data = await websocket.receive_text()
            message = json.loads(data)

            result = rag.query(message['question'])

            await websocket.send_json({
                "type": "response",
                "answer": result['answer'],
                "sources": result['sources']
            })
    except Exception as e:
        await websocket.send_json({"type": "error", "detail": str(e)})
```

```
        await websocket.close()
```

Listing 7.2: RAG chat endpoint

Chapter 8

Agentic AI Implementation

8.1 LangChain Agent Setup

Create ai_service/agent.py:

```
from langchain.agents import Tool, initialize_agent, AgentType
from langchain.chat_models import ChatOpenAI
from langchain.callbacks import StreamingStdOutCallbackHandler
import os

class SynapseAgent:
    def __init__(self):
        self.llm = ChatOpenAI(
            openai_api_key=os.getenv('OPENAI_API_KEY'),
            model_name="gpt-3.5-turbo",
            streaming=True,
            callbacks=[StreamingStdOutCallbackHandler()]
        )
        self.tools = self._setup_tools()
        self.agent = initialize_agent(
            self.tools,
            self.llm,
            agent=AgentType.STRUCTURED_CHAT_ZERO_SHOT_REACT_DESCRIPTION,
            verbose=True
        )

    def _setup_tools(self):
        """Setup agent tools"""
        tools = [
            Tool(
                name="Search Knowledge Base",
                func=self.search_kb,
                description="Search the knowledge base for articles and papers"
            ),
            Tool(
                name="Generate PDF Report",
                func=self.generate_pdf,
                description="Generate a PDF report with findings"
            ),
            Tool(
                name="Create Project",
                func=self.create_project,
                description="Create a new project in the system"
            ),
```

```
        ]
        return tools

    def search_kb(self, query: str) -> str:
        """Search knowledge base"""
        # Implementation
        return "Results from knowledge base"

    def generate_pdf(self, title: str, content: str) -> str:
        """Generate PDF using ReportLab"""
        from reportlab.lib.pagesizes import letter
        from reportlab.pdfgen import canvas

        filename = f"reports/{title}.pdf"
        c = canvas.Canvas(filename, pagesize=letter)
        c.drawString(100, 750, title)
        c.drawString(100, 730, content)
        c.save()
        return f"PDF generated: {filename}"

    def create_project(self, name: str, description: str) -> str:
        """Create project"""
        return f"Project '{name}' created successfully"

    def run(self, user_input: str) -> str:
        """Run agent"""
        return self.agent.run(user_input)
```

Listing 8.1: ReAct agent setup

Chapter 9

Frontend Implementation

9.1 Next.js Project Setup

```
cd frontend
npm create next-app@latest . --typescript

# Install additional dependencies
npm install axios zustand recharts next-themes
```

Listing 9.1: Next.js setup

9.2 API Client Configuration

Create `frontend/lib/api.ts`:

```
import axios, { AxiosInstance } from 'axios';

const API_URL = process.env.NEXT_PUBLIC_API_URL || 'http://localhost:8000/api';

const api: AxiosInstance = axios.create({
  baseURL: API_URL,
  headers: {
    'Content-Type': 'application/json',
  },
});

// Auth interceptor
api.interceptors.request.use((config) => {
  const token = localStorage.getItem('access_token');
  if (token) {
    config.headers.Authorization = `Bearer ${token}`;
  }
  return config;
});

// Response interceptor
api.interceptors.response.use(
  (response) => response,
  (error) => {
    if (error.response?.status === 401) {
      localStorage.removeItem('access_token');
      window.location.href = '/login';
```

```
    }
    return Promise.reject(error);
  }
);

export default api;
```

Listing 9.2: API client with auth

9.3 State Management with Zustand

Create `frontend/store/authStore.ts`:

```
import create from 'zustand';
import api from '@/lib/api';

interface User {
  id: string;
  email: string;
  is_premium: boolean;
}

interface AuthStore {
  user: User | null;
  isLoading: boolean;
  login: (email: string, password: string) => Promise<void>;
  logout: () => void;
  fetchUser: () => Promise<void>;
}

export const useAuthStore = create<AuthStore>((set) => ({
  user: null,
  isLoading: false,

  login: async (email: string, password: string) => {
    set({ isLoading: true });
    try {
      const response = await api.post('/auth/token/', { email, password });
      localStorage.setItem('access_token', response.data.access);
      const userRes = await api.get('/auth/profile/');
      set({ user: userRes.data });
    } finally {
      set({ isLoading: false });
    }
  },

  logout: () => {
    localStorage.removeItem('access_token');
    set({ user: null });
  },

  fetchUser: async () => {
    try {
      const response = await api.get('/auth/profile/');
      set({ user: response.data });
    } catch (error) {
      set({ user: null });
    }
```

```
  },
}));
```

Listing 9.3: Auth store with Zustand

9.4 Pages Structure

Create key pages in `frontend/pages/`:

```
// pages/dashboard.tsx
import { useEffect } from 'react';
import { useAuthStore } from '@/store/authStore';
import { useRouter } from 'next/router';

export default function Dashboard() {
  const { user, fetchUser } = useAuthStore();
  const router = useRouter();

  useEffect(() => {
    if (!user) {
      fetchUser();
    }
    if (!user) {
      router.push('/login');
    }
  }, []);

  return (
    <div className="min-h-screen bg-gray-50 dark:bg-gray-900">
      <div className="container mx-auto px-4 py-8">
        <h1 className="text-3xl font-bold text-gray-900 dark:text-white">
          Welcome, {user?.email}
        </h1>
        {user?.is_premium && (
          <p className="text-green-600 dark:text-green-400">Premium Member</p>
        )}
      </div>
    </div>
  );
}
```

Listing 9.4: Dashboard page

Chapter 10

Automation System Implementation

10.1 Workflow Model and Task Queue

Create backend/automation/models.py:

```
from django.db import models
from django.contrib.auth import get_user_model
import uuid

User = get_user_model()

class Workflow(models.Model):
    STATUS_CHOICES = [
        ('draft', 'Draft'),
        ('active', 'Active'),
        ('paused', 'Paused'),
        ('completed', 'Completed'),
    ]

    id = models.UUIDField(primary_key=True, default=uuid.uuid4)
    user = models.ForeignKey(User, on_delete=models.CASCADE)
    name = models.CharField(max_length=255)
    description = models.TextField(blank=True)
    status = models.CharField(max_length=20, choices=STATUS_CHOICES, default='draft')
    config = models.JSONField() # Workflow configuration
    schedule = models.CharField(max_length=100, blank=True) # Cron expression
    created_at = models.DateTimeField(auto_now_add=True)
    updated_at = models.DateTimeField(auto_now=True)

    class Meta:
        ordering = ['-created_at']

class WorkflowExecution(models.Model):
    RESULT_CHOICES = [
        ('pending', 'Pending'),
        ('running', 'Running'),
        ('success', 'Success'),
        ('failed', 'Failed'),
    ]

    id = models.UUIDField(primary_key=True, default=uuid.uuid4)
    workflow = models.ForeignKey(Workflow, on_delete=models.CASCADE)
    status = models.CharField(max_length=20, choices=RESULT_CHOICES)
    result = models.JSONField(default=dict)
```

```
    error_message = models.TextField(blank=True)
    started_at = models.DateTimeField(auto_now_add=True)
    completed_at = models.DateTimeField(null=True)
```

Listing 10.1: Workflow automation model

10.2 Celery Workflow Execution

Create `backend/automation/tasks.py`:

```
from celery import shared_task
from django.core.mail import send_mail
from .models import Workflow, WorkflowExecution
from datetime import datetime

@shared_task
def execute_workflow(workflow_id: str):
    """Execute workflow and handle email notifications"""
    try:
        workflow = Workflow.objects.get(id=workflow_id)
        execution = WorkflowExecution.objects.create(
            workflow=workflow,
            status='running'
        )

        # Execute workflow logic
        result = process_workflow(workflow.config)

        execution.status = 'success'
        execution.result = result
        execution.completed_at = datetime.now()
        execution.save()

        # Send notification
        send_workflow_notification(workflow.user.email, workflow.name, result)

    except Exception as e:
        execution.status = 'failed'
        execution.error_message = str(e)
        execution.save()

def process_workflow(config: dict) -> dict:
    """Process workflow configuration"""
    results = {}
    for step in config.get('steps', []):
        if step['type'] == 'scrape':
            results[step['name']] = scrape_data(step['source'])
        elif step['type'] == 'process':
            results[step['name']] = process_data(step['input'])
    return results

def send_workflow_notification(email: str, name: str, result: dict):
    """Send email notification"""
    send_mail(
        f'Workflow "{name}" Completed',
        f'Your workflow has completed successfully.\n\nResults: {result}',
        'noreply@synapse.ai',
        [email],
```

```
        fail_silently=False,
    )
```

Listing 10.2: Celery workflow tasks

Chapter 11

Testing Strategy

11.1 Backend Unit Tests

Create backend/tests/test_models.py:

```
import pytest
from django.test import TestCase
from users.models import User
from articles.models import Article
from datetime import datetime

@pytest.mark.django_db
class TestUserModel(TestCase):
    def setUp(self):
        self.user = User.objects.create_user(
            email='test@example.com',
            password='testpass123'
        )

    def test_user_creation(self):
        assert self.user.email == 'test@example.com'
        assert self.user.is_active

@pytest.mark.django_db
class TestArticleModel(TestCase):
    def setUp(self):
        self.article = Article.objects.create(
            title='Test Article',
            content='Test content',
            source='hackernews',
            source_url='https://example.com/article',
            published_date=datetime.now()
        )

    def test_article_creation(self):
        assert self.article.title == 'Test Article'
        assert self.article.source == 'hackernews'
```

Listing 11.1: Django model tests

11.2 API Integration Tests

Create backend/tests/test_api.py:

```python
import pytest
from rest_framework.test import APIClient
from rest_framework import status
from users.models import User

@pytest.mark.django_db
class TestAuthAPI:
    def setup_method(self):
        self.client = APIClient()
        self.user = User.objects.create_user(
            email='test@example.com',
            password='testpass123'
        )

    def test_token_obtain(self):
        response = self.client.post('/api/auth/token/', {
            'email': 'test@example.com',
            'password': 'testpass123'
        })
        assert response.status_code == status.HTTP_200_OK
        assert 'access' in response.data

    def test_profile_view(self):
        response = self.client.post('/api/auth/token/', {
            'email': 'test@example.com',
            'password': 'testpass123'
        })
        token = response.data['access']

        self.client.credentials(HTTP_AUTHORIZATION=f'Bearer {token}')
        response = self.client.get('/api/auth/profile/')

        assert response.status_code == status.HTTP_200_OK
        assert response.data['email'] == 'test@example.com'
```

Listing 11.2: API endpoint tests

11.3 Frontend Tests with Jest

Create `frontend/__tests__/authStore.test.ts`:

```typescript
import { useAuthStore } from '@/store/authStore';
import axios from 'axios';

jest.mock('axios');
const mockedAxios = axios as jest.Mocked<typeof axios>;

describe('Auth Store', () => {
  beforeEach(() => {
    jest.clearAllMocks();
  });

  it('should login user successfully', async () => {
    mockedAxios.post.mockResolvedValue({
      data: { access: 'test_token' }
    });
```

```
    const store = useAuthStore.getState();
    await store.login('test@example.com', 'password');

    expect(localStorage.getItem('access_token')).toBe('test_token');
  });

  it('should logout user', () => {
    const store = useAuthStore.getState();
    store.logout();
    expect(localStorage.getItem('access_token')).toBeNull();
  });
});
```

Listing 11.3: Frontend store tests

Chapter 12

Code Quality and Deployment

12.1 Linting and Formatting

Configure `backend/setup.cfg`:

```
[flake8]
max-line-length = 100
exclude = .git,__pycache__,venv,migrations
ignore = E203,W503

[isort]
profile = black
line_length = 100

[mypy]
python_version = 3.11
warn_return_any = True
strict_optional = True
```

Listing 12.1: Python linting configuration

Configure `frontend/.eslintrc.json`:

```
{
  "extends": "next/core-web-vitals",
  "rules": {
    "react/no-unescaped-entities": "warn",
    "@next/next/no-html-link-for-pages": "off"
  }
}
```

Listing 12.2: ESLint configuration

12.2 Pre-commit Hooks

Create `.pre-commit-config.yaml`:

```
repos:
  - repo: https://github.com/pre-commit/pre-commit-hooks
    rev: v4.4.0
    hooks:
      - id: trailing-whitespace
      - id: end-of-file-fixer
```

```yaml
    - id: check-yaml

  - repo: https://github.com/psf/black
    rev: 23.3.0
    hooks:
      - id: black
        language_version: python3.11

  - repo: https://github.com/PyCQA/flake8
    rev: 6.0.0
    hooks:
      - id: flake8

  - repo: https://github.com/PyCQA/isort
    rev: 5.12.0
    hooks:
      - id: isort
```

Listing 12.3: Pre-commit hooks configuration

12.3 Continuous Integration with GitHub Actions

Create `.github/workflows/test.yml`:

```yaml
name: Tests

on: [push, pull_request]

jobs:
  backend-tests:
    runs-on: ubuntu-latest
    services:
      postgres:
        image: postgres:15
        env:
          POSTGRES_PASSWORD: postgres
        options: >-
          --health-cmd pg_isready
          --health-interval 10s
          --health-timeout 5s
          --health-retries 5

    steps:
      - uses: actions/checkout@v3

      - name: Set up Python
        uses: actions/setup-python@v4
        with:
          python-version: '3.11'

      - name: Install dependencies
        run: |
          cd backend
          pip install -r requirements.txt

      - name: Run tests
        run: |
          cd backend
```

```
        pytest tests/ --cov=synapse

  frontend-tests:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v3

      - name: Set up Node
        uses: actions/setup-node@v3
        with:
          node-version: '20'

      - name: Install dependencies
        run: |
          cd frontend
          npm install

      - name: Run tests
        run: |
          cd frontend
          npm run test
```

Listing 12.4: GitHub Actions CI pipeline

12.4 Deployment Checklist

Before deploying to production:

1. Update `DEBUG=False` in production `.env`
2. Generate new `SECRET_KEY` using Django secret key generator
3. Configure database backups and replication
4. Setup monitoring with Prometheus and Grafana
5. Enable SSL/TLS certificates
6. Configure AWS S3 for static and media files
7. Setup SendGrid for email notifications
8. Run database migrations on production
9. Perform load testing before launch
10. Setup log aggregation (CloudWatch/ELK)
11. Configure error tracking (Sentry)
12. Setup uptime monitoring and alerts

Chapter 13

Monitoring and Observability

13.1 Prometheus Metrics

Create `backend/synapse/prometheus.py`:

```
from prometheus_client import Counter, Histogram, Gauge
import time

# Define metrics
request_count = Counter(
    'app_requests_total',
    'Total requests',
    ['method', 'endpoint', 'status']
)

request_duration = Histogram(
    'app_request_duration_seconds',
    'Request duration',
    ['endpoint']
)

active_users = Gauge(
    'app_active_users',
    'Number of active users'
)

db_connection_pool = Gauge(
    'app_db_connections',
    'Active database connections'
)

celery_tasks = Counter(
    'app_celery_tasks_total',
    'Total Celery tasks',
    ['task_name', 'status']
)
```

Listing 13.1: Prometheus metrics setup

Bibliography

[1] Django Software Foundation. (2023). *Django 4.2 Documentation*. Retrieved from https://docs.djangoproject.com/

[2] FastAPI. (2023). *FastAPI - Modern Web Framework*. Retrieved from https://fastapi.tiangolo.com/

[3] LangChain. (2023). *LangChain Documentation*. Retrieved from https://langchain.readthedocs.io/

[4] Vercel. (2023). *Next.js Documentation*. Retrieved from https://nextjs.org/docs

[5] PostgreSQL Global Development Group. (2023). *PostgreSQL 15 Documentation*. Retrieved from https://www.postgresql.org/docs/

[6] Docker Inc. (2023). *Docker Documentation*. Retrieved from https://docs.docker.com/